
HackPit

Full Capability Assessment & Agreed Plan

Date: 26 July 2026

Scope: backend · frontend · pipeline · docker · docs · knowledge base

Measured against: OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB/CTF · bug bounty · client pentests

Contents

HackPit — Capability Assessment & Build Log

0. Verdict

1. Remaining gaps

1.1 The PTY gap — now closed, without losing the transcript

1.2 The human-approval model — settled, not open

2. Feature-by-feature audit

Companion (KB / search / attack paths)

Cockpit (execution)

AD graph + orchestration

Detection footprint

Code scan (SAST)

Frontend

3. Knowledge base assessment

Findings

Recommended ingestion, in priority order — status

4. Source-usage audit — "did I use my sources properly?"

4.1 The headline finding: the pipeline only reads *.md

4.2 Consolidation is lossy for some sources

4.3 The biggest missed idea — PentestGPT's Pentest Task Tree — is now BUILT

4.4 The structural pattern

5. What is genuinely excellent — do not refactor

6. Security posture of HackPit itself

7. What the assessment examined

PART II — WHAT REMAINS

Standing policy (unchanged, still governs the project)

Explicitly deferred (with the reason)

reconFTW — recon gaps mined, and what to skip

PART III — BUILD LOG (Phases 1–5, shipped 2026-07-26)

Decisions taken (D1–D17)

Phase 1 — reach a real target

Phase 2 — make it remember

Phase 3 — make it fast to drive

Phase 4 — content & goal-specific

Phase 5 — the PTY gap, and finishing the KB

Windows execution backend — the WinRM driver (shipped 2026-07-26)

Post-assessment refinements (2026-07-27)

reconFTW review + incorporation (2026-07-27)

Persistence / backdoors (TA0003) enrich (2026-07-27)

AV/EDR evasion + traffic obfuscation (2026-07-27)

Gate-integrity audit (2026-07-27)

Gate integrity — build #5 (2026-07-27)

Verification

Status

HackPit — Capability Assessment & Build Log

Date: 2026-07-26 **Scope:** every backend module, the frontend, the pipeline, the docker stack, the docs, and the live KB **Targets this is measured against:** OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB boxes/CTF · bug bounty · real-world client pentests.

Status note (updated). This began as an assessment (Part I) and an agreed plan (Part II). **All four phases of that plan have since been built**, and a fifth followed — the execution substrate (Phase 1), the state model (Phase 2), the drive-speed gaps (Phase 3), the content-and-goal-specific work (Phase 4: KB Route-B ingest, the evasion/OPSEC channel, the HTTP repeater, pivot/tunnel routing, exam-mode report templates), and **Phase 5**, which closed the two things Part I had left standing: a real PTY (added as a *second* surface, so the auditable one survives) and the recommended KB enrichment (PortSwigger Academy, HackTricks Cloud, the HTB slice, and a CVE→exploit index). Part I has been trimmed to what genuinely remains; the completed blockers, decisions and phases live in **Part III — Build Log**.

0. Verdict

The **architecture is genuinely strong** — the safety engineering and the LLM-grounding discipline are better than most commercial tooling.

The original problem this assessment found was *reach*: nothing in HackPit was fake or stubbed, but the container it executed into contained seven programs, so it could not finish a real HTB box, touch a real AD domain, run a bug-bounty recon sweep, or complete an OSCP-style engagement. **That execution substrate has now been rebuilt (Phase 1), the loop reasons over structured state instead of stdout tails (Phase 2), the drive-speed gaps are closed (Phase 3), the content/goal work is done (Phase 4), and the interaction and knowledge gaps Part I left standing are closed (Phase 5).** See Part III.

Since then **Phase 5** closed the PTY gap (a real terminal as a *second* surface, keeping the auditable one) and finished the KB enrichment the assessment recommended, and the **Windows execution backend** (a WinRM driver) has since made the CRTP toolset, the OSCP AD set and real internal pentests *executable* — and made the existing AD attack-path graph run **live** instead of on synthetic data. What remains is a short list of *deliberately deferred* items — a VPS for blind callbacks, and the last few thin KB categories — each deferred for a stated reason, not left undone. The Windows/CRTP execution target is **no longer among them**: it is closed by the WinRM driver against an external VM you run (see the Windows execution backend in Part III and D9). Risk-tiered approval is also gone — **decided against** (see D5). See §1 and Part II's "Explicitly deferred".

1. Remaining gaps

The blocker sections for everything the five phases fixed have been removed (see Part III). What is left is genuinely small.

1.1 The PTY gap — now closed, without losing the transcript

This section previously read "no true PTY — deliberate partial", and framed it as a trade: readable logged transcripts *or* full-screen tooling. **Phase 5 rejected that trade.** The premise was wrong — the two are not alternatives if they are separate surfaces.

`:kali` is unchanged: still a persistent shell delimiting each command with a sentinel over a plain pipe, still producing the clean, escape-free, per-command transcripts reports are built from. Alongside it, `:terminal` allocates a **real PTY** in the same open sandbox, so `vim`, `top`, an interactive `msfconsole`, a raw `evil-winrm` shell and `python -c 'pty.spawn'` upgrades all render. Both are audited; both carry identical containment. See Part III, Phase 5.

1.2 The human-approval model — settled, not open

Current state: every command needs individual `approved=true`; anything the danger heuristic flags additionally needs `dangerous_ack`; `:kali` and `:terminal` are human-driven (typing *is* the approval). Phase 3 added **one-keystroke approve** (`Enter` = approve · `S` = skip · `Esc` = stop; a dangerous command never fires on `Enter` alone).

This section previously proposed **risk-tiered approval** — auto-running "passive" commands, batch-approving a plan of read-only ones — as open work. **That is now decided against** (D5). Per-command approval is the project's single load-bearing safety property in engagement mode: Wall A is down, the sandbox reaches the internet, the host and the LAN, and nothing else bounds *where* a command can go. A tier that auto-runs anything would remove the only gate on the one mode that needs it most, and it would do so by trusting a classifier — a new component, on the wrong side of the safety boundary, to decide when the human can be skipped. The convenience it buys is a keystroke that has already been optimised away.

Per-command approval is therefore **standing policy, not a deferred item**, and does not appear in the deferred list. The prerequisite work that would have enabled tiering (`_looks_like_host()`, Phase 3 step 12) was worth doing on its own merits — the scope check is trustworthy now regardless. **Unchanged and non-negotiable: never remove the gate for anything the danger heuristic flags.**

2. Feature-by-feature audit

Companion (KB / search / attack paths)

Component	State	Notes
KB load + serve	Solid	1,601 entries in memory, exclusions dropped at the door and re-filtered defensively at query time.
Hybrid search (<code>pipeline/search.py</code>)	Solid	BM25 (stdlib) + <code>nomic-embed-text</code> cosine, weighted RRF (lex 1.0 / vec 0.5 — correct call for a KB full of exact identifiers), tier boost, whole-query title bonus. Degrades to lexical when Ollama is down instead of 500-ing.
Attack-path composition	Solid, genuinely novel	Writeup-first mode + KB-first mode. Retrieval seeded per-phase so no phase starts empty. Step eligibility filtering (<code>is_step_eligible</code>) is unusually careful — excludes writeups, defensive-hardening guides, tool-install meta-docs, personal logs, grab-bags.

Component	State	Notes
Grounding / validation	Best-in-class	Cited <code>entry_id</code> s must resolve or the step downgrades to <code>ai_suggested</code> ; commands come from the KB entry, never the model; <code>target_adaptation</code> is dropped entirely if it names an FQDN not in the supplied facts.
<code>substitute_target</code>	Excellent	CIDR sentinel to avoid double-rewrite, host-position gating so <code>.env.example</code> and <code><script></code> survive, refuses to rewrite what it can't rewrite confidently.
<code>foreign_refs</code>	Excellent	Honesty marker — reports a foreign host/AD domain rather than guessing a replacement. Right call.
Channel-2 context grounding	Good	Writeup/methodology content as reasoning background, strictly separated from the step pool, with a leak guard over model output.
Engagement sessions	Works	SQLite, per-step checked + pasted results, debounced autosave.
Report generation	Very good, now multi-template (Phase 4)	Evidence built programmatically , not by the LLM — the model was observed mis-transcribing a port number. Exam/format templates: OSCP (per-host walkthrough + a proof.txt table spliced from state), CPTS (exec-summary + findings register), H1/Bugcrowd (impact-first + a CVSS 3.1 score <i>computed</i> , not asserted). proof.txt/local.txt are real per-host state (auto-captured when a command reads a flag file, or pasted). Collision-proof fences; lab vs REAL-TARGET labelled; detection footprint appended grounded-only; optional red-team OPSEC roll-up (D10).
Assistant chat	Works	Session-aware, KB-grounded, deterministic citation extraction (scans the reply for ids that resolve).
Scripts arsenal	Works	1,028 scripts deduped from the KB across 6 groups.

Cockpit (execution)

Component	State	Notes
Gated executor	Excellent design, now properly toolled	Four ordered gates, mode split, argv-only (never a shell), SSE streaming, run records persisted. The image it execs into now ships ~63 of the 73 catalogued tools (Phase 1).
Isolation gate	Real	<code>assert_isolation_proven()</code> structurally inspects every attached network for <code>internal: true</code> . Not a comment — an actual check.
Engagement mode	Correct	Explicit entry required, fail-closed on unknown/exited id, per-command approval re-checked even on the prevalidated path (belt-and-suspenders in <code>iter_run</code>).
Scope model (<code>scope.py</code>)	Good	Hosts, <code>*.wildcards</code> , CIDRs, <code>!exclusions</code> , <code>*</code> opt-out. Fail-closed. No DNS at match time (avoids the "resolve an out-of-scope name to decide it's out of scope" leak). The <code>_looks_like_host()</code> false-positives are fixed (Phase 3).
Recon-driven expansion	Good	Mines run output for hosts, sorts by scope, in-scope join the live allowed set, out-of-scope surfaced read-only. Cannot widen the scope. Capped and never silently truncating.
Orchestrator loop	Now state-grounded	Proposes one command, never executes. Feeds on the structured state model + live task tree (Phase 2) instead of stdout tails.

Component	State	Notes
Engagement state model	New (Phase 2)	hosts/services/endpoints/credentials/findings, upsert-only, fed by output parsers + loot-file ingest; drives the planner and a UI panel. Executes nothing (AST-asserted).
<code>:kali</code>	Now a persistent shell (Phase 3)	One long-lived <code>docker exec -i sh ; cd /env/background</code> jobs persist. Same containment (hardcoded open container, human-only, audited, no isolation gate). Deliberately no PTY — that is what keeps its transcripts clean (§1.1).
<code>:terminal</code>	Real PTY, second surface (Phase 5)	<code>pty.fork()</code> inside the same open box, driven over a framed WebSocket to xterm.js; full-screen tools render and resize. Containment mirrors <code>:kali</code> point for point; the raw stream is audited. Does not replace <code>:kali</code> (§1.1).
<code>:exploits</code>	New (Phase 5)	Version-keyed CVE → exploit lookup over the sandbox's local exploit-db catalogue — 47k exploits, 25k distinct CVEs. Read-only; executes nothing.
Live sessions	Well-designed, now toolled	Start is a gated command; stdin is human-only and source-scan locked.
HTTP repeater	New (Phase 4)	Compose/send/replay/diff. Argv-only curl inside the hardcoded open box (no shell parses a request field; body on stdin), human-only + source-scan locked like <code>:kali</code> , scope-checked against a named engagement, every send run-recorded.
Pivot / tunnels	New (Phase 4)	chisel/ligolo-ng lifecycle (human-only start/stop) + pure route resolution and a visible proxychains rewrite applied <i>before</i> the approval screen. A tunnel's subnet enters scope only via an explicit, audited amendment; recon expansion still cannot widen.

AD graph + orchestration

Component	State	Notes
BloodHound parser	Genuinely good	Handles v4/v5/CE, zip/dir/json/bytes/mapping, reconciles naming drift, synthesizes DCSync from <code>GetChanges</code> + <code>GetChangesAll</code> , emits coverage warnings for missing collection methods. Real work.
Path engine	Correct	BFS shortest path over abusable edges only, abuse-rank tie-break, k-shortest-ish alternatives.
Technique catalog	Good	25 edge kinds, KB-grounded with catalog fallback, target-type specialization (<code>GenericAll</code> on a group → <code>AddMember</code>). Each edge now also carries a native Windows variant (PowerView/Rubeus/Mimikatz) for live WinRM execution alongside the Linux impacket/evil-winrm one.
Orchestrator	Excellent safety design	The model picks an edge index , never a command. Cannot invent a host, cannot author a command, cannot reach an edge outside the collection. A pick outside the list is refused rather than repaired. Now runs live — the approved command executes over WinRM on a selected Windows target (proposes, never auto-fires; regression-locked for the WinRM path too).
<code>advance</code> endpoint	Excellent	Advancement requires a <code>run_id</code> that was approved and exited 0 , verified server-side — transport-agnostic, so the WinRM path advances the walk identically.
Execution tooling	Now present + executes live	impacket, certipy-ad, bloodyad, netexec, evil-winrm, bloodhound.py, kerbrute, responder, mitm6 are in the image (Phase 1). The WinRM driver (Windows execution backend) now runs the abuse live against a real Windows/AD box you run in VMware — the graph is no longer synthetic-only. Live verification against a real box is deferred until that VM is up (build + unit tests are hermetic, mocking WinRM).

Detection footprint

Read-only, honest, well-sourced. Real SigmaHQ rule UUIDs, real ATT&CK ids (Enterprise v19.1), and a verifier script (`pipeline/detection_sources.py --verify`) that re-checks every id against live upstream. The "describes what blue sees, never how to be seen less" line is enforced in code. **Scale:** ~133 command specs, 19 distinct ATT&CK techniques — a good purple-team *flavour*, not a coverage tool.

CRTP conflict — now reconciled (Phase 4, D10). CRTP includes AMSI/logging evasion, which the panel used to refuse outright. It now has an **additive second channel**: the blue-team detection view is byte-for-byte unchanged and still passes the never-prescribe guard, and a separate, opt-in `opsec` channel adds the offensive half — what makes a command loud, the quieter tradecraft, and, mandatorily, *what still records it*. The OPSEC channel has its own guard (it may never advise disabling/clearing/tampering with a sensor); the LLM may extend it for uncatalogued commands, marked `ai_suggested` . See Part III, Phase 4.

Code scan (SAST)

Well-built and genuinely safe. Static-only invariant asserted at every `_spawn()` and again by `test_codescan_safety.py` . Deliberately orthogonal — imports nothing from the engagement/executor/scope model. **Limits — now widened (post-assessment):** the offline bundle grew from 19 rules (Python/JS/TS) to **34 across 8 languages** — Java/Go/PHP/Ruby/C# added (`rules/hackpit-languages.yaml` : command injection, SQLi, unsafe deserialization, code-eval, file-inclusion, SSRF), plus a **ruleset picker** (bundled / per-language / a registry pack for the full online catalogue). Still offline-first. See "Post-assessment refinements".

Frontend

16 routes, real API wiring throughout, SSE streaming, no mocked data layer (`cockpitSample.ts` is the only sample content, explicitly labelled). New in Phases 1–3: the arsenal availability band, per-run time-budget + detach controls, the engagement-state/task-tree panel, the credential-vault "use" action, and the persistent `:kali` shell UI. **Gaps:** no global current-target/engagement context, no engagement export/import, no multi-target view, and the accepted 10-error `react-hooks` lint baseline (documented; `next build` passes exit 0).

3. Knowledge base assessment

2,617 entries (1,601 at the time of the assessment; +66 in Phase 4, +950 in Phase 5) · **median body 3,000 chars.**

Source distribution:

Source	Entries
hacktricks	715 (45%)
madstuff	260
some-hacking-resources	233
writeups	59
payloadsallthethings	49
claudе-bug-bounty	46

Source	Entries
peh-notes	43
htb-writeups	41
clauder-red	39
oscp-cpts-notes	36
decepticon	33
htb-academy	13
hackpit-authored	12
htb-box-pdfs	9
galaxy-checklist	7
htb-my-resources	5
shodan-dorks	1

Phase 5 added: hacktricks-cloud 578 · portswigger 372.

Tiers: 2,265 tier-3 · 241 tier-2 · 111 tier-1 (your own) — see finding 2. **Ranking rebalanced (post-assessment):** tier is now a *substance-gated* nudge, not a blanket trust prior — a thin tier-1 stub no longer outranks richer, more relevant lower-tier content, and a command-rich page wins close calls regardless of tier. See "Post-assessment refinements".

Findings

1. **PayloadsAllTheThings payload depth — now recovered (Phase 4, D11/D12).** PATT's 66 `.txt` payload lists + the shodan dork list are ingested at **payload-level granularity** (64 `payload-set` + 2 `dork-list` entries, `no_merge`-guarded so consolidation can never collapse them again), each a searchable entry over a full sidecar corpus mounted read-only at `/payloads`; 56 `oscp_tools` files went to the Scripts Arsenal. KB 1,601 → 1,667. See Part III, Phase 4.
2. **"Grounded in your own notes" is mostly "grounded in HackTricks."** Search boosts tier-1 (`search.py`: 58), but with 111 tier-1 entries that lever has little to pull on. **Growing tier-1 is the highest-leverage KB work still available — still open**, and now the *only* open KB item. Phase 5 grew the KB by 950 entries, all tier-3, so the ratio moved the wrong way: the fix is writing, not ingesting.
3. **Empty categories — mostly fixed (Phase 5).** Was: cloud 2 · mobile 2 · iot 2 · forensics 1 · ics 1 · phishing 1 · supply-chain 1. Now **cloud 535** and **supply-chain 47** (HackTricks Cloud), and web/API coverage is deep (PortSwigger). **Still thin:** mobile · iot · forensics · ics · phishing — none of which is on the target list, so they stay unfilled by choice rather than oversight.
4. **HTB Academy — narrowed, not closed (Phase 5).** The local folder is an 18-module *slice*, not the CPTS syllabus, and HTB content is proprietary so nothing is scraped. Auditing the slice against what had been ingested found two files silently lost — one tracked in git but absent from disk, one lost purely for having a `.txt` extension — both now recovered. Closing this properly needs the course itself.
5. **Missing as retrievable topics — fixed (Phase 5).** API security (GraphQL/REST), OAuth/SSO, race conditions, business logic, request smuggling, prototype pollution, deserialization and SSRF→cloud-metadata chains all

now resolve, most to PortSwigger Academy material with the lab that drills each one alongside.

6. **No CVE/exploit index — fixed (Phase 5).** :exploits answers service+version → CVE → public exploit over the sandbox's own exploit-db catalogue, version-compared rather than substring-matched.

Item 2 is the only KB item still open. Items 3–6 were the recommended next batches; they were never in scope for the first four phases and were built in Phase 5.

Recommended ingestion, in priority order — status

1. **PortSwigger Web Security Academy** — the single best web-security source, and structured. **Done (Phase 5, +372).**
2. **HTB Academy modules properly (CPTS syllabus).** **Partially — the local slice is fully ingested; the syllabus needs the course.**
3. **HackTricks Cloud.** **Done (Phase 5, +578).**
4. **PayloadsAllTheThings re-ingested at payload-level granularity (see §4).** **Done (Phase 4).**
5. **An exploit-db / CVE mapping keyed on service+version.** **Done (Phase 5) — and deliberately not as KB prose; see Part III.**

4. Source-usage audit — "did I use my sources properly?"

Measured on disk against the built KB, not inferred.

4.1 The headline finding: the pipeline only reads *.md

pipeline/ingest.py:229 → `sorted(source_path.rglob("*.md"))`; pipeline/ingest_notes.py:599 likewise.
Every .txt, .json, .py, .sh, .csv, .yaml file in every source tree was invisible to the pipeline.

Source folder	Files	.md	KB entries	Non-md skipped	Of which is <i>real content</i>
hackdic	807	806	715	1	—
madstuff	798	797	260	1	—
some hacking resources	233	233	233	0	—
PayloadsAllTheThings	481	134	49	347	66 .txt payload lists + 28 .py scripts
oscp-cpts-notes	288	85	36	203	174 PNG screenshots (+ git internals)
oscp_tools	99	1	0	98	28 .exe binaries, ~22 .ps1 / .sh / .py scripts
HTB_academy	48	16	13	32	3 PNGs — effectively nothing
shodan-dorks	32	1	1	31	1 .txt dork list
Hacking-Tools	31	1	0	30	nothing — git internals only

Source folder	Files	.md	KB entries	Non-md skipped	Of which is <i>real content</i>
PRACTICAL ETHICAL HACKING NOTES	110	45	43	65	images
more new resources (writeups)	118	72	100	46	images
htb my resources	5	5	5	0	—
htb boxes	10 (pdf)	0	9	—	PDF path handled separately
PentestTools	0	0	0	—	empty folder

Scale: stripping git internals, images and compiled binaries, the genuinely lost knowledge was roughly **95 files** — chiefly **PATT's 66 .txt payload lists** (the pipeline ingested the *explanations* and discarded the *payloads* — backwards for bug bounty), the **shodan-dorks .txt**, and **~22 oscp-tools scripts**. **Fixed (Phase 4, D11/D12):** an additive Route-B ingester (`pipeline/ingest_corpora.py`) folded all 66 payload lists + both dork lists into the KB as `payload-set` / `dork-list` entries with full sidecar corpora, and 56 `oscp-tools` files into the Scripts Arsenal — without rewriting a single existing entry (verified byte-identical) and idempotently. Two env traps hit and handled: `rglob` silently omitted 2 of 66 files (OneDrive dehydration → union with `git ls-files`) and 22 files were AV-locked (→ `git-blob` recovery).

4.2 Consolidation is lossy for some sources

`madstuff` 797 md → 260 (33%). `oscp-cpts-notes` 85 md → 36 (42%). `PayloadsAllTheThings` 134 md → 49 (37%). Some is correct deduplication into HackTricks; a 3:1 collapse on your OSCP/CPTS notes is worth an eyeball — those are exactly the entries you'd want surviving as tier-1/tier-2.

Audited (post-assessment) — healthy dedup, no loss. A read-only provenance audit of the OSCP set: **36 survived as their own entries (33 with real commands), 12 folded into stronger canonical pages** (`PayloadsAllTheThings/peh-notes/some-hacking-resources` — preserved as `also_covered_in` + variants, not deleted), and the remaining ~37 were **same-technique OS/lab splits merged together** (the GitBook ships Linux/Windows/lab variants as separate files). **Zero** OSCP entries were dropped as low-value (only 11 were dropped KB-wide). Also corrected: `oscp-cpts-notes` is a **tier-3 public GitBook repo**, not your own writing — your genuine tier-1 comes from `writeups` / `peh-notes` / `htb-my-resources`. **Decision: no re-ingest** — re-adding with `no_merge` would just recreate duplicate technique pages that hurt search. See "Post-assessment refinements".

4.3 The biggest missed idea — PentestGPT's Pentest Task Tree — is now BUILT

The assessment's single highest-value reasoning-layer recommendation was PentestGPT's task tree: a *persistent, hierarchical, status-tracked engagement state the model reads from and writes back to every turn*. **This shipped in Phase 2** (`backend/state/tasks.py`) — layered `1 / 1.1 / 1.1.1` tasks with `todo | done | n/a`, updated as results arrive via model-proposed operations that code validates. It is retained here as the rationale for what was built. (Implementation note: HackPit's variant has the model return *validated operations* rather than rewriting the whole tree, so one bad response cannot silently rewrite the plan.)

4.4 The structural pattern

The pipeline is excellent at *techniques* and lossy on *lists, workflows and payload corpora*. A checklist is a *sequence*, a dork list is a *set*, a payload collection is a *corpus* — all three were forced through a technique-shaped, markdown-only schema and collapsed. **Both fixes shipped (Phase 4):** the Route-B ingester accepts non-markdown inputs, and the second entry shapes (`meta.kind = payload-set / dork-list`, `no_merge`-guarded) survive consolidation intact. The `checklist` shape is defined and available; no checklist source was in the D12 scope, so none were ingested yet.

5. What is genuinely excellent — do not refactor

- **The safety architecture is real.** Source-scan regression locks on `:kali` and session stdin, four ordered gates, clean lab/engagement mode split, `assert_isolation_proven()` as a structural runtime check. Better than most commercial tooling. (The Phase 1–3 work preserved every one of these invariants; the test suite grew with it.)
 - **The grounding discipline is the best idea in the project.** Grounded-vs-`ai_suggested` labelling; `entry_ids` must resolve or the step is downgraded; commands come from the KB, never the model; **evidence spliced programmatically into reports rather than written by the LLM**; `foreign_refs` as an honesty marker; the Channel-2 leak guard.
 - **The AD orchestrator picks an EDGE, not a command.** A safety design that also improves output quality. The Phase-2 task tree follows the same principle (validated ops, not free-form rewrites).
 - `advance` requires an approved, exit-0 run — evidence-based state transition, verified server-side.
 - **The consolidation pipeline** — alias-key + cosine match, structural merge, idempotent, provenance preserved.
 - **The detection footprint's framing and sourcing** — real Sigma UUIDs with a drift verifier.
 - `substitute_target` — CIDR sentinel, host-position gating, refusal to rewrite what it can't rewrite confidently.
 - **The BloodHound parser** — v4/v5/CE reconciliation with honest coverage warnings.
-

6. Security posture of HackPit itself

- **No authentication anywhere.** No `Depends`, no key, no session. CORS restricted to `localhost:3000`. Honestly documented in code.
 - On a laptop this is fine. **On a VPS attack box it is an unauthenticated RCE endpoint** — `POST /cockpit/kali` (and the new persistent-shell routes) run arbitrary `sh -c` in a container with full network reach to your host and LAN. Auth is a hard blocker before any non-localhost deployment.
 - Secrets handling is correct: `llm_config.json` gitignored, API key written to disk but never returned, collector preview redacts the password/hash. The engagement-state DB (which now holds **real captured credentials**, by decision) is gitignored like the rest.
 - The repo is code-only; KB, sources, sessions DB and `.env` are all gitignored. A prior full-history audit confirmed no secrets were ever committed.
 - **Known:** a self-scan flagged SSRF and CORS misconfiguration in HackPit's own backend (memory obs #1496). Worth revisiting before any exposure.
-

7. What the assessment examined

Read in full: every non-test backend module (executor, allowlist, config, sandbox, engagement, scope, kali, session, runstore, router, models; attack_path, main, orchestrator, llm, report, chat, context_channel, sessions; all of adgraph, arsenal, detection, codescan), `pipeline/search.py` , `docker/Dockerfile.sandbox` , `docker-compose.yml` , the QA report, and the live KB.

Structurally mapped: the frontend (routes, API surface, component→endpoint wiring, demo/mock scan — none found beyond the labelled `cockpitSample.ts`); the test files; `pipeline/consolidate.py` .

Source trees inspected on disk: `hexstrike-ai` (151 `@mcp.tool` defs), `mantishack` , `Decepticon` , `PentestGPT` (task-tree prompt read directly), `claude-red` , `claude-bug-bounty` , `Galaxy-Bugbounty-Checklist` , `hackdic` , `htb boxes` , the writeup trees, `some hacking resources` . File counts and markdown ratios measured per tree.

Verified live (assessment): container tooling (37-tool probe), SYN-scan capability, nuclei templates, container capabilities, KB statistics, ingester file-type globs.

PART II — WHAT REMAINS

Decided with Zaid, 2026-07-26. All five phases and every decision that drove them are in Part III. The build is complete; this section is only what is intentionally left — some of it deferred, some of it rejected outright.

Standing policy (unchanged, still governs the project)

These are not open work — they are the rules the project runs by, and building is finished, so they no longer need a decision table. Kept here as the active policy:

- **D2 — Windows + Docker Desktop; a VPS is added later, only for bug-bounty callbacks.** Docker Desktop is a real Linux VM (WSL2); the one thing it can't give is a publicly reachable listener for blind SSRF/XXE/RCE. The repeater (Phase 4) sends and reads the *direct* response; the VPS piece is still deferred until bounty work needs it.
- **D5 — Per-command approval stays. Risk-tiering is rejected, not deferred.** One-keystroke approve shipped (Phase 3). Tiering — auto-running "passive" commands, batch-approving read-only plans — was reconsidered and **decided against** (§1.2): in engagement mode per-command approval is the *only* thing bounding where a command may go, and a classifier deciding when to skip the human is a new component on the wrong side of that boundary. Every execution surface since preserves this — repeater, tunnels (rewrite *visible before* approval), and Phase 5's `:terminal` (human-only, no agent path).
- **D8 — HexStrike is a reference, never an execution backend.** Its tools bypass all four gates, the target-lock, the scope model and the audit trail. Rejected, not deferred.
- **D9 — CRTP Windows execution: CLOSED via a WinRM driver against an external VM (was "accept the gap").** The original decision accepted that CRTP's PowerShell/.NET tooling can't run on the Linux container. That gap is now closed the honest way: **no Windows VM lives inside the project** — HackPit *drives* one you run in VMware over WinRM (Model A), a new execution transport behind the same gates. Windows-only tools now report runnable when a Windows target is selected (reconciled per active target); on a Linux run they are still N/A and marked. The AD graph executes live against that target. See the Windows execution backend in Part III.

Explicitly deferred (with the reason)

- **A VPS for blind out-of-band callbacks (D2)** — a NAT'd laptop has no public listener; deferred until bounty work needs it.
- **A Windows execution target for CRTP (D9) — DONE** (no longer deferred): the WinRM driver executes CRTP/AD work live on an external VMware VM you run; only the live-box browser verification is deferred until that VM is up.
- **Growing tier-1 (§3, finding 2)** — the only KB item still open, and the one that cannot be solved by ingesting: it needs Zaid's own writing.
- **The HTB Academy syllabus proper (§3, finding 4)** — the local slice is fully ingested; the rest is proprietary and needs the course.
- **Thin categories with no target on the list** — mobile · iot · forensics · ics · phishing (§3, finding 3). Unfilled by choice.

- **C2-framework install + the persistence evasion/OPSEC half — DONE** (build #4, no longer deferred): Sliver is installed and wired to a gated panel, and the persistence OPSEC notes were backfilled. Empire and weeveily remain catalogued reference-only by choice.
- **Live-fire verification of the C2 and tunnel surfaces (build #4) — OUTSTANDING, and the largest known gap in that build.** Everything shipped in build #4 is verified *hermetically* (42 suites, 459 checks, docker/subprocess stubbed) and *through the panel*, plus an image smoke test that invokes each binary. What has **never** happened is an end-to-end run against real infrastructure. Specifically still to do:
 - **Catch a live Sliver beacon.** The server lifecycle starts and stops and is audited, but no implant has ever called back. Needs a listener reachable from a victim host — i.e. the same VPS gap as D2, or a lab VM on a routable segment. Until then the implant argv is only as correct as the catalog's documented syntax; a wrong flag surfaces as `status='failed'`, not as a containment failure.
 - **Stand up a real DNS tunnel.** `dnscat2-server` and `iodined` start inside the engage sandbox and the client one-liner is handed back, but no delegated zone exists to test against. Needs an authoritative zone the operator controls with an NS record pointed at the listener. `iodined` also needs a TUN device in the engage sandbox — plausible given its `NET_ADMIN`, but unproven.
 - **Run a generated artifact on a Windows box and watch the telemetry.** The evasion engine emits artifacts and the paired footprints claim specific Event IDs and Sysmon codes. Those claims are transcribed from ATT&CK v19.1 and SigmaHQ, not observed here. The honest version of this feature is only fully honest once someone detonates an artifact on an instrumented host and confirms the footprint matches what actually landed in the log. This is the natural pairing with the D9 Windows VM. None of this blocks the build shipping — the containment properties are what the test suite locks, and those are provable without live infrastructure. But the *efficacy* claims are, until then, documented rather than demonstrated, and this list should be worked through before build #4 is described as field-ready.
- **An independent review of build #4's Tasks 8–13 — DONE** (no longer outstanding). Run in a fresh session (the safety classifier had begun refusing subagent dispatch partway through the build, and it refuses on accumulated context). It found **no containment hole** — no agent path, no shell, the container never request-influenced, no delivery primitive — and five substantive defects, all fixed: a multi-technique request that built one artifact and described a different one; the T1690/T1685 mis-mapping above; two stub headers describing memory patching when the stubs are managed-reflection bypasses; a no-agent-path test whose positive control counted the wrong variable (99 files reported, 5 actually scanned); and a `_gate_request` comment asserting a scope-gate behaviour that does not hold. Plus seven minor items — an image smoke test that could not fail, unpinned installs, dead fields, weak assertions. Every fix carries a test that fails without it.
- **Gate-integrity Criticals 2 and 3 — DONE (build #5, 2026-07-27).** Both closed; full detail in Part III. **Critical 2** (the WinRM first-token classification — the sharpest known hole in the tool, because it defeated the red-confirm by moving a cmdlet one token to the right on the one path that reaches a real domain-joined host) is fixed by classifying the whole joined PowerShell script, deliberately without parsing it, with the gate and the transport now deriving that string from one shared function. **Critical 3** (source-scan locks covering a fraction of the tree) is fixed by one shared scanner: whole-tree `rglob`, repo-relative allow-lists, an AST pass for indirection, and per-lock planted-violation controls — coverage per lock goes 30 of 69 modules to 67. The plan's third task, the one that matters longest, is written down in `backend/AGENTS.md`. **One item from the same audit remains open: I2, the ungated tunnel listener start** — `cockpit/tunnels.start_tunnel` reaches `subprocess.Popen` without `validate_request`, and `POST /cockpit/tunnels` has no `approved / dangerous_ack` field. It was outside build #5's scope.
- **Pinning the build-#4 install layer** (audit M2) — three of five install paths are unpinned (`git clone` of `dnscat2`, four `gem install`s, `pip install donut-shellcode`). `docker/pin-build4-versions.sh` resolves the real

values and rewrites the layer; a test fails if the unpinned set grows. Deliberately left unpinned rather than pinned to invented values.

- **Still open from that review:** three installs in the build-#4 image layer (dnscat2's clone, four Ruby gems, donut-shellcode) remain **unpinned**, so the layer is not byte-reproducible. Pinning needs upstream values resolved from the network; `docker/pin-build4-versions.sh` resolves them and rewrites the layer in place. The gap is declared in the Dockerfile and asserted by `test_evasion.py` , which fails if the unpinned set grows.
- **Kali VM as an execution target; HexStrike as an execution backend; risk-tiered / batch approval — rejected, not deferred** (D8, D5, §1.2).
- **Authentication before any non-localhost deployment** (§6) — no decision needed until a VPS enters the picture, but a hard blocker the moment it does. `:terminal` makes this sharper: an unauthenticated *interactive terminal* onto a box that reaches the host and LAN.
- Screenshot capture, recon diffing/monitoring, checklist-driven runs.

reconFTW — recon gaps mined, and what to skip

reconFTW (six2dez, MIT) was reviewed for what HackPit's *recon* was missing — its thinnest area (the incorporation narrative is in Part III). Respecting the constraint that governs everything here — HackPit is human-gated, so "incorporate" means adopt reconFTW's **knowledge**, never its auto-runner — the concrete gaps, where each landed, and rough effort:

- **URL-triage pipeline** — `gf` + `qsreplace` + `unfurl` (arsenal `web`) + a KB methodology entry. The single biggest bug-bounty gap: HackPit harvested URLs but had no triage step. *Low — done.*
- **Subdomain permutations** — `gotator` , `regulator` , `subwiz` (arsenal `recon`). An entire enum technique HackPit had none of. *Low-med — done.*
- **Mass resolution** — `puredns` + `massdns` + `dnsvalidator` , with a baked resolver list (arsenal `recon` + `image`). HackPit had only `dnsx` . *Med — done.*
- **JS mining** — `jsluice` , `subjs` , `getjs` (arsenal `web`): endpoint/secret extraction, not just crawling. *Low-med — done.*
- **ASN → CIDR expansion** — `asnmap` , `mapcidr` (arsenal `recon`) for wide-target scope work. *Low — done.*
- **External OSINT** — `trufflehog` , `github / gitlab-subdomains` , `dorks_hunter` , `gitdorks_go` , `msftrecon` , in a new `osint` category (HackPit had no OSINT home). *Low — done.*
- **Bucket/cloud enum** — `s3scanner` , `cloud_enum` (arsenal `cloud`): external asset discovery, distinct from the posture scanners. *Low-med — done.*
- **Injection completers** — `commix` , `SSTImap` , `nomore403` , `crlfuzz` (arsenal `web`): fill the cmd-injection / SSTI / 403-bypass / CRLF tool gaps (skills existed; tools didn't). *Low — done.*
- **Recon ordering** — the subdomain-enum sequence and the URL→gf-triage→fuzz pipeline, as two KB `checklist` entries (`recon-methodology-*`) + an advisory note in the composer's real-target prompt. The higher-leverage half — tools without the ordering is the cheap half. *Low-med — done.*

Skip — conflicts with the human-gated model (rejected, not deferred): the `reconftw.sh` runner and `reconftw.cfg` engine (autonomous chaining is the exact thing HackPit rejects), `interlace` (parallel command execution), `notify` (autonomous alerts), monitor/incremental/diff auto-rescan, the axiom/Ax fleet, `reconftw_ai`

and faraday. `inscope` is redundant — `scope.py` is stronger (fail-closed, no DNS at match time).
`interactsh` /OOB stays deferred on the VPS (D2, unchanged). `waymore` was considered and left out: `gau` +
`waybackurls` + `katana` already cover URL harvesting.

PART III — BUILD LOG (Phases 1–5, shipped 2026-07-26)

Branch `sandbox-kali-image` . Full hermetic safety suite green throughout (36 test files); both Docker proofs 4/4 (lab still egress-less; engage fully open); browser-verified with Ollama (`qwen3:8b`).

Decisions taken (D1–D17)

Every decision that drove the build. D1/D3/D4/D6/D7 were the Phase-1 build; D9 was a standing accepted gap that is **now built** (the Windows execution backend); D2/D5/D8 are standing policy (Part II); D10/D11/D12 were Phase-4 work, now built; D13 is this document; D14/D15 shaped Phase 5; **D16/D17 are build #4** — D16 amends D10 and is the one policy reversal in the project's history.

- **D1 — Fix the sandbox image; HackPit is the attack box.** A Kali VM's advantages (root, raw sockets, VPN, `/etc/hosts`) are all reachable in Docker via capabilities + `/dev/net/tun` . *Built (Phase 1).*
- **D3 — Capabilities + root on the ENGAGE sandbox only.** Lab keeps `cap_drop: ALL` + non-root. *Built.*
- **D4 — All three sandboxes get the new toolset; only the lab's *network* isolation stays.** *Built.*
- **D6 — Catalog describes reality, not aspiration.** *Built.*
- **D7 — Startup reconciliation check.** *Built.*
- **D9 — CRTP Windows execution.** Was "accept the gap; no Windows VM." Now **closed by the WinRM driver** (Windows execution backend): HackPit drives an external VMware VM you run — no Windows VM inside the project. Windows-only tools run when a Windows target is selected; still N/A + marked on a Linux run. *Built (the AD graph executes live).*
- **D10 — Allow evasion/OPSEC content as an additive second channel, keeping the blue view.** *Built (Phase 4).*
- **D11 — Fix the KB gap via Route B (additive merge), not a rebuild.** *Built (Phase 4).*
- **D12 — Ingest PATT payloads + shodan dorks into the KB; `oscp_tools` into the Scripts Arsenal.** *Built (Phase 4).*
- **D13 — Plan lives in this document.** *Done.*
- **D14 — The PTY is a SECOND surface, never a replacement.** The old framing treated auditable transcripts and full-screen tooling as alternatives. They are only alternatives on one surface. `:kali` keeps its sentinel; `:terminal` gets the pty; both are audited and identically contained, and a test asserts `kali.py` never grows a pty. *Built (Phase 5).*
- **D15 — The CVE index is a lookup table, not a KB batch.** "Find the exploit for this version" wants an exact, deterministic table; embedding it as prose would make the one query it exists to answer fuzzy. Its own data shape, search path and surface — and it executes nothing. *Built (Phase 5).*
- **D16 — Lift the OPSEC sensor-tamper ban; the honesty marker becomes the sole invariant.** Amends D10. The OPSEC channel previously refused any sensor-blinding phrasing, which became incoherent once build #4 shipped an engine that emits AMSI-patch and ETW-blind artifacts: the tool would have been producing artifacts it was forbidden to describe. The ban is lifted **entirely**, and two invariants carry the whole weight

instead — every note must name what **still records** the activity, and the blue-view footprint is always produced alongside and can never be suppressed. The blue-side describe-never-prescribe guard and all four execution gates are **unchanged**. A posture shift on a public repo, taken deliberately. *Built (build #4).*

- **D17 — Split the gating for C2 rather than gate it uniformly.** Server and listener **lifecycle** is human-only with no red-confirm (operator infrastructure on the operator's own sandbox — no target is touched, so clicking start is the approval); **artifact generation** is a fully gated command (approval + scope + red-confirm) because it produces something that will run on someone else's machine. Uniform gating would have meant either a meaningless confirm on every server start or no confirm on a payload build. *Built (build #4).*

Phase 1 — reach a real target

Commits `e4e8de5`, `80a18d5`, `2d0928f`.

- **New sandbox image** (`docker/Dockerfile.sandbox`) on `kalilinux/kali-rolling` + `kali-linux-headless`, with SecLists, the Kali `wordlists` set (rockyou unzipped) and **~13,400 nuclei templates** baked in (the lab has no egress, so nothing can be fetched at runtime). Thematic tool layers for the specific tools the assessment found missing — `impacket`, `netexec`, `evil-winrm`, `bloodhound.py`, `certipy-ad`, `bloodyad`, `responder`, `mitm6`, `smbmap`, `enum4linux-ng`, `gobuster/nikto/feroxbuster/subfinder/httpx/amass`, `metasploit`, `exploitdb`, `hashcat/john`, `chisel/ligolo-ng/proxychains/socat/ncat`, `openvpn` — plus `katana/kerbrute/waybackurls/dalfox/gau/rustscan/jwt_tool` as pinned release binaries (no Kali package). **Result: ~63 of 73 catalogued tools present.** Kali's `setcap'd nmap / fping` are stripped at build so `no-new-privileges` doesn't refuse to exec them.
- **Privilege split** (`docker-compose.yml`) — the **engage** sandbox runs `user: root` with Docker's default capability set + `NET_ADMIN` and `/dev/net/tun`, so `-sS / -sU / -O`, `masscan`, `tcpdump`, `privileged-port binds` (`responder/ntlmrelayx`) and `VPN` all work. **Lab and :kali are untouched** — `uid 1000`, `cap_drop: ALL`, no devices. Verified: `engage` does `SYN` scans and binds `:445`; `lab/ :kali` refuse `-sS`.
- **Per-request timeouts** (default 180s, ceiling 3600s) on `/cockpit/exec` and `:kali`; **background/detached jobs** returning a `run_id` and a reconnectable replay stream (`/cockpit/runs/{id}/stream`), with a reaper. `:kali`'s old hardcoded 60s is gone.
- **Loot volume** — `backend/data/engagements` → `/loot` on the `engage` and `:kali` sandboxes (deliberately **not** the isolated lab); each engagement works in `/loot/<id>` via `docker exec -w`, so `nmap -oA` output survives `docker compose down`.
- **Startup reconciliation** (`GET /tools`) — probes the running sandbox with one `command -v` sweep over every catalogued name+alias, filters the planner's prompt to installed tools, and surfaces the gaps in the arsenal UI. Windows-only tools (`rubeus/powerview/mimikatz/winpeas`) are marked `platform: windows`, kept for planning/write-ups, and never proposed.
- **Arsenal catalog** reconciled against the shipped image; the orchestrator prompt no longer hardcodes `gobuster / nikto`.

Phase 2 — make it remember

Commits `c606f6d` (backend), `a8f40ae` (UI).

- **backend/state/ package** — the structured engagement state the assessment identified as the deepest gap: `hosts` → `services`, `endpoints`, `credentials`, `findings` as dataclasses over a shared SQLite store where

every write is an upsert (re-running a scan corrects, never duplicates). Output **parsers** (nmap XML, httpx/ffuf/nuclei JSON, secretsdump) as pure functions, and a post-run **ingest** that reads both a run's stdout and any file it wrote into its loot directory (that is how `nmap -oA` populates state). The package **executes nothing** — AST-asserted, because ingest runs automatically after every command.

- **Pentest Task Tree** (`state/tasks.py`) — PentestGPT's central idea (§4.3): layered `1 / 1.1 / 1.1.1` tasks with `todo | done | n/a`, persistent per session, updated as results arrive. The model returns **validated operations** (`add_subtask / mark_done / mark_na / ...`) that code applies against the stored tree; one bad op is rejected on its own and the rest still apply.
- **Orchestrator grounding** — the proposer prompt now leads with the accumulated state + live task tree, with raw output demoted to a short tail. Measured: **460 chars of state vs 6,600 of stdout tails** for 12 runs of one nmap, and each fact appears once instead of once per run. A tail window forgets; state does not.
- **Credentials in the prompt: real values** (Zaid's explicit decision) so the planner writes fully-formed commands — one constant (`render.INCLUDE_CREDENTIAL_SECRETS`) with a test proving the flip works, so the choice stays reversible.
- **UI** — the `CockpitState` panel: counters, the task tree (with "seed from plan"), hosts+services grouped, endpoints with status colours, credentials masked with per-row reveal + validated/untested, findings by severity.
- **Bonus fix found by the browser pass** — the arsenal `/arsenal` response model was silently dropping `platform / runs_here`, so every tool badged "windows only". Fixed + regression-tested.

Phase 3 — make it fast to drive

Commit `b316b74`.

- **Step 11 — one-keystroke approve.** The guided loop takes `Enter` = approve, `S` = skip, `Esc` = stop, hints shown on the buttons. A dangerous proposal never fires on `Enter` (the explicit danger confirm is still required); shortcuts are ignored while a text field is focused.
- **Step 12 — `_looks_like_host()` fixed.** Flipped from "assume host unless the last segment is a known file extension" to a **positive test**: a dotted token is a host only if its last label is a plausible alphabetic TLD and not a file extension. Stops the false rejections (`directory-list-2.3-medium`, `Mozilla/5.0`, `-oA scan.1.2`, version strings) while still catching real hosts; a genuine off-target host is still refused. This was the prerequisite for any future auto-run tier.
- **Step 13 — persistent `:kali` shell.** One long-lived `docker exec -i sh` instead of a fresh exec per command; each command is delimited over the pipe with a per-command sentinel so output and exit code read back cleanly. `cd /env/background` jobs persist (verified in-browser: `cd /tmp → export F00=... → echo cwd=$(pwd) foo=$F00` returned `cwd=/tmp foo=...` across three separate API calls). Every `:kali` containment invariant intact. No PTY, by design (§1.1). (Caught the Windows `\n → \r\n` stdin-translation bug in the process — the Linux shell was receiving `pwd\r`; fixed with bare-LF stdin.)
- **Step 14 — credential vault.** Captured credentials fill the `<user> / <password> / <ntlm-hash> / <domain>` placeholders the AD templates carry, one click instead of retyping a hash. The placeholder→field mapping lives server-side (`state/credvault.py`) so front and back can't drift; a password fills `<password>` not `<hash>` and vice-versa; only credential placeholders are touched. Verified in-browser: "use `svc_sql`" turned `nmap -u <user> -p <password> ...` into `nmap -u svc_sql -p S3cr3t! ...`.

Phase 4 — content & goal-specific

Commits `90fe260`, `885d776`, `bf46695`, `8486c56`, `937c420`.

- **Item 1 — KB Route B additive ingest (D11/D12).** `pipeline/ingest_corpora.py` — a targeted ingester over only the missing files, run *additively* on the built KB (never through the real pipeline, which would revert downstream enrichment and rewrite a 15 MB gitignored, Defender-sensitive file). Two regression-locked guarantees: existing entries pass through as **bytes** (no key/escape drift on the 1,601 it never looked at), and every line it owns carries `meta.corpus_ingest` so a re-run is byte-identical. Shapes (§4.4): `64 payload-set + 2 dork-list` entries, all `no_merge` so consolidation can't collapse a corpus into a technique page; each holds a capped excerpt while the full list is a **sidecar** under `data/kb/payloads/`, mounted read-only at `/payloads` in all three sandboxes so ffuf/wfuzz point straight at it. `56 oscp_tools` files went to the Scripts Arsenal as file-backed rows (a path to copy, not 20,000 lines of PowerView), 38 Windows-only ones kept and marked `runs_here=false` (D9). Two env traps handled: rglob's OneDrive omission (→ union with `git ls-files`) and AV-locked files (→ git-blob recovery). KB 1,601 → 1,667; arsenal 1,028 → 1,137. The Defender exclusion on `HackPit\data` was added first.
- **Item 2 — evasion/OPSEC as an additive second channel (D10).** The blue-team detection view is **byte-for-byte unchanged** and still passes `assert_describes_not_prescribes` (a test proves every blue field is identical with and without the offensive half attached, and that the default `footprint()` carries no `opsec` key — so tagging/reports/run-annotation are untouched). `detection/catalog.py` gains a curated **OPSEC** table (10 notes keyed by spec key: what makes it loud, the quieter tradecraft, the tradeoff, and — mandatory — `still_recorded`, the honesty marker). `resolver.py` gains an opt-in `include_opsec` channel with its **own** guard (`assert_opsec_is_separate`): a note that fabricates the honesty marker or advises disabling/clearing/tampering with a sensor is rejected. `report.py` gains an off-by-default red-team OPSEC roll-up. The panel renders it as a distinct amber section. LLM may extend to uncatalogued commands, `ai_suggested`.
- **Item 3 — HTTP repeater.** `cockpit/repeater.py` mirrors `:kali` containment (hardcoded open container, human-only + source-scan locked, audited) and adds a scope check `:kali` lacks. **Argv-only curl**, never a shell — method/headers/URL are discrete tokens, the body rides on stdin, so a header of `a; rm -rf /` is one literal `-H`. A human clicking Send is the approval (no per-send prompt); the orchestrator/agent have zero path to `send`. When the send names an active engagement the URL host is scope-checked (out-of-scope → 403, nothing sent). Frontend `/repeater`: compose/send, response view, per-session history with edit-to-resend and a line-level body diff. The VPS-for-callbacks piece stays its own decision (D2).
- **Item 4 — pivot/tunnel routing.** `cockpit/tunnels.py` — chisel/ligolo-ng lifecycle (start the listener in the engage sandbox, hand back the agent one-liner to paste on the compromised host; human-only start/stop, source-scan locked). `route_for` / `wrap_command` are **pure** — they compute a **visible** `proxchains -q` prefix (chisel/SOCKS) or leave the command unwrapped with a route note (ligolo/interface), applied *before* the approval screen so the human approves the exact argv (silent routing was rejected for exactly this reason). A tunnel's subnet enters scope only via `engagement.add_pivot_subnet` — the one deliberate, audited widening path; recon expansion still cannot widen. Frontend `/tunnels`: start form, live tunnels with the copy-ready one-liner + per-subnet "add to scope (by hand)", and a pure route preview.
- **Item 5 — exam-mode report templates + proof.txt as per-host state.** `state` `Host` gains `local_txt / proof_txt + ownership()`; captured automatically when a command reads a flag file (`proof.txt / root.txt` → `proof`; `local.txt / user.txt` → `local` — a stray 32-hex hash is never mistaken for a flag) or pasted via `POST /sessions/{id}/state/proof`; the state panel badges foothold vs owned. `report.py` gains four templates — standard, **OSCP** (per-host walkthrough + a `proof.txt` table spliced from state at

{{PROOF_TABLE}} , computed not model-written), CPTS (exec-summary + findings register), H1/Bugcrowd (impact-first + a CVSS 3.1 base score computed by cvss31_base , matching the official calculator). Every template keeps the shared grounding rules + the {{EVIDENCE}} splice. Frontend: a template picker on the report screen.

Phase 5 — the PTY gap, and finishing the KB

Commits e8eeef , 8f0b91b (terminal), 84e31bf (CVE index), f6fb6a4 (KB batches).

- **Item 1 — a real PTY, as a SECOND surface.** §1.1 used to call this a trade: readable transcripts or full-screen tooling. It is not, if they are separate surfaces. :kali is untouched — still sentinel-delimited over a plain pipe, still the clean per-command transcripts reports are built from — and cockpit/terminal.py adds :terminal beside it. docker exec -t refuses without a client TTY and a WebSocket handler has none, so the pty is allocated **inside** the container by a constant driver that pty.fork() s bash and multiplexes it over the pipes. Its stdin is **framed** (0x00 = keystrokes, 0x01 = COLSxROWS → ioctl TIOCSWINSZ), which is what makes resize possible without an in-band escape a keystroke could forge; its stdout is the raw pty stream, straight to xterm.js. Containment mirrors :kali point for point: hardcoded container (the request carries session_id / cols / rows and nothing else), driver a raw-string constant passed as one argv element, no isolation gate, **human-only and source-scan locked** like run_kali , and the raw stream audited to the run store — capped, and checkpointed every 30s so a crash still leaves a transcript. The WS route pins origin by hand, because CORS middleware does not cover WebSockets. test_terminal.py (16 checks) locks all of it *plus the additive property itself*: kali.py must still carry its sentinel and must never grow a pty, so the clean transcript cannot quietly die. Verified live — tty reports /dev/pts/0 , top and vim render, resize propagates 30×100 → 43×132 → 60×200.
- **Item 2 — CVE → exploit index (§3, finding 6).** The OSCP inner loop, built as a **keyed lookup rather than another KB batch**: "find the exploit for *this* version" wants an exact table, not prose retrieval, so it gets its own data shape, search path and surface. pipeline/ingest_exploitdb.py reads /usr/share/exploitdb/files_exploits.csv out of the sandbox image — 47,108 exploits, 27,384 with a CVE, 25,041 distinct — nothing fetched from the internet. exploit-db titles follow <Product> <Version> - <Vuln> on ~100% of rows, so parsing the left side turns a flat catalogue into (product, version) → exploits with the constraint typed (exact / lte / range / wildcard / none); 32,807 rows yield a version. backend/exploits/ then does what searchsploit 's substring match cannot: 2.4.49 satisfies < 2.4.50 , sits inside a range, and matches the 2.4.x line — each with a *different stated verdict*, and the verdict is the ranking's primary key, with token similarity only breaking ties inside a tier. (Blending them let Apache 2.4.x outrank the entry naming 2.4.49 exactly — caught on real data, now regression-locked.) Surfaces: /exploits , plus a per-service exploits → link in the state panel, so a fingerprinted service reaches its exploits in one click. Every hit names the file **already inside the sandbox**. It executes nothing, and test_exploits.py asserts the package contains no subprocess path and never reaches the execution layer — finding is not running.
- **Item 3 — PortSwigger Web Security Academy (+372).** pipeline/fetch_portswigger.py turns the site into the same shape every other source has, so nothing downstream is special-cased. URLs come from the publisher's own sitemap.xml rather than from crawling — it is complete (the all-topics page renders client-side, so scraping it yields 19 of ~140) and nothing is ever guessed; robots.txt restricts only /bappstore/bapps/download/ . 398 pages, 273 of them labs, whose Solution steps survive. Two rules written by what dry runs actually did: **labs and sub-topic pages are no_merge** (the first run folded ten Academy pages — Reflected/Stored/DOM XSS, contexts, CSP, dangling markup — into one pre-existing grab-bag called "xss resource", so "Reflected XSS" stopped being retrievable as itself; only a topic root may consolidate, which the

Academy expresses as URL depth, taking 90 merges down to 26); and the two cheat sheets use `<section>` not `<main>`, so without a fallback they were 2 of 18 pages silently extracting zero bytes.

- **Item 4 — HackTricks Cloud (+578), and the cloud gap closed.** Same GitBook layout and adapter as the HackTricks book already ingested. Two corrections, both from dry runs: it is **not class-grouped** (grouping collapsed 44 distinct CI/CD pages — Jenkins RCE, GH Actions cache poisoning, Okta, Cloudflare — into one candidate), and `CANON` gains only **technique-level** cloud classes. "kubernetes", "ci-cd" and "supply-chain" name a *platform*, not a technique, and `CANON` is the authority on what consolidates; including them collapsed every "Kubernetes X" page into one entry. Their absence is now documented in `CANON` so they are not re-added. **cloud 2 → 535, supply-chain 1 → 47.**
- **Item 5 — the HTB Academy slice, audited (§3, finding 4).** Auditing what the local folder holds against what had been ingested found two files silently lost, both to one root cause — bare `rglob` over `*.md`. `File_Inclusion/README.md` (4.5 KB of LFI module content) is tracked in git but **gone from disk**, the dehydration/quarantine pattern this repo has hit before — and that module is full of web-shell examples, which is exactly what trips the signature. `File_Upload_Attacks/1.txt` (8 KB of module answers and webshell walkthroughs) was lost purely for its extension: §4.1's headline finding in miniature. Now `_all_md` (`rglob U git ls-files`, with git recovery) plus `.txt`.

KB 1,667 → 2,617, 0 malformed rows, embeddings rebuilt incrementally, scripts index rebuilt, `entries.jsonl` verified present and counted after every pass (the Defender-quarantine trap).

Windows execution backend — the WinRM driver (shipped 2026-07-26)

Commits `72b0959` (*transport + profiles + windows mode*), `cf64b8f` (*profile CRUD/picker + per-target /tools*), `d8854dc` (*native Windows abuse variants + AD live*), `1ce53ce` (*frontend*), the VM guide, and this doc. Closes D9 the honest way. See `docs/WINDOWS-EXECUTION.md` + `docs/WINDOWS-TARGET-SETUP.md`.

- **A new execution transport, behind the same gates.** `docker exec` is swapped for a WinRM call; nothing about the safety model changes. A third mode, `windows` (alongside `lab` and `engagement`), is selected by `ExecRequest.windows_profile_id`. The target is the profile's **host** — **hardcoded server-side, never a request field** (the same containment shape `:kali` gets from its hardcoded container: a command physically cannot reach a box you did not pick). Gates: `windows` (profile exists) → `target` (host in the engagement scope, if one is also named) → **never-auto-run** approval → danger red-confirm. No isolation gate — it is a real external box, like engagement.
- **Model A — HackPit drives, it does not own.** No Windows VM lives inside the project. The operator runs a Windows/AD VM in VMware Workstation; HackPit opens a WinRM session and runs one PowerShell command string on the box (Rubeus/PowerView/Mimikatz/.NET all run *there*). `pywinrm` is **lazy-imported** so the hermetic suite needs no dependency and no network; the AD-live path is unit-tested with a **mocked** transport.
- **Saved connection profiles** (`cockpit/winprofiles.py`) — a "Windows targets" store in the gitignored `sessions.db`: name, host, transport (WinRM; SSH a documented later seam), port, username, `password` or `ntlm-hash` (pass-the-hash, presented `LM:NT`), domain. The secret is **write-only** — masked to `has_secret` in every view, read only by the transport, never in a response, record or command line. A **captured vault credential can fill a profile**, resolved server-side so the secret never round-trips.
- **The AD graph executes live.** Every abusable edge gained a **native Windows variant** (PowerView/Rubeus/Mimikatz) beside the Linux one; the walk/orchestrator's proposed command runs over WinRM when a Windows target is picked, and `advance` still requires an approved + `exit-0` run. The danger heuristic learned the native destructive cmdlets (`Set-DomainUserPassword`, `Add-DomainObjectAcl`, `Set-`

DomainObjectOwner , Add-DomainGroupMember , Set-DomainRBCD , Invoke-Mimikatz , ...), so a native DCSync/password-reset trips the **same** red confirm as its Linux cousin — the oracle test now checks both transports.

- **Per-target tool reconciliation.** Windows-only tools (Rubeus/PowerView/Mimikatz/winPEAS) report runnable when a Windows target is selected and N/A on a Linux run — reconciled per active target (`/tools?windows_profile_id=...`), never a global flip. Served from `main.py` so the cockpit stays arsenal-blind.
- **Frontend.** A **Windows targets** page (`/windows`) — profile CRUD, masked secrets, connectivity test — and a "run on" picker in the AD walk (Linux sandbox vs a WinRM target); the confirm/flags/command shown follow whichever command will actually run.
- **Safety regression-lock** (`test_winrm_safety.py`): host-locked to the profile / no gate bypass / secrets never leak / **the orchestrator cannot auto-run WinRM** (transport reachable only from the gated executor + the human router probe, source-scanned). Functional path (`test_winrm.py`) uses a mocked transport.

Deferred to a live VM: the live/browser verification against a real Windows box, until the operator's VM is up (build + unit tests are hermetic) — the same way tunnels and AD-live execution were deferred.

Post-assessment refinements (2026-07-27)

Three items from a read-through of this assessment, each done the low-risk way. All query-time / config / ranking changes — **no KB re-ingest, no 15 MB rewrite, no Defender risk.**

- **KB consolidation audited (\$4.2) — no re-ingest needed.** A read-only provenance audit confirmed the OSCP "3:1 collapse" is healthy dedup, not loss: 36 command-rich survivors, 12 folded into stronger canonical pages (still attributed), ~37 same-technique OS/lab splits merged, **0 dropped as low-value**. Also corrected the framing — `oscp-cpts-notes` is a tier-3 public GitBook repo, not the author's own writing. Decision: leave the KB untouched (re-adding with `no_merge` would recreate duplicate technique pages that hurt search).
- **Relevance-first KB ranking (\$3, tiers).** The composer was already relevance-first (BM25 + cosine RRF; tier a small additive nudge), but a *thin* tier-1 stub still got a flat boost that could edge out richer, more relevant lower-tier content. Fixed in `pipeline/search.py`: the tier-1 boost is now **gated on substance** (a stub with no commands + little body gets nothing), and a **completeness nudge** (command-rich, any tier, saturating/capped) lets content decide close calls. Trust breaks ties among substantive entries; it no longer manufactures relevance. Locked by `test_search_ranking.py`.
- **SAST coverage widened (\$ Code scan).** The offline Semgrep bundle grew from 19 rules (Python/JS/TS) to **34 across 8 languages** — Java/Go/PHP/Ruby/C# added (`rules/hackpit-languages.yaml`) — plus a **ruleset picker** (bundled / per-language, or a registry pack for the full online catalogue). Default scan loads the whole offline directory. Locked by `test_codescan_rules.py` (rules well-formed + 8-language coverage + `semgrep -validate` , resolved from the venv the runner uses so it validates rather than skips). The new rules were confirmed **valid** (semgrep `--validate` : 0 errors, 34 rules) and to **fire** on real vulnerable samples (Java/Go/Ruby/C#). One robustness fix fell out of that check: on some Windows builds semgrep itself *crashes* on a `.php` file, and the scan treated semgrep as fatal — so a crash now **degrades to a warning** (the scan still returns with whatever else ran, like bandit), instead of 502-ing the whole scan. (*Engagement export/import was considered and dropped — the existing report generator already covers a shareable dump.*)
- **UI cleanup (Frontend).** Small polish from a working session: the two shell tiles were merged to **one :kali tile that opens the real PTY** (`/terminal`) — full-screen tools render from the everyday shell, while the sentinel "transcript shell" (clean per-command records) is preserved off-nav at `/kali` ; both backends and their containment tests stay intact. The **phishing** category is now surfaced on the front-page grid (its KB entries

already existed; forensics/ics/supply-chain stay hidden). The **top nav** dropped the redundant `:Library` tile (home is the wordmark), the search field was widened to a one-line field, the unused accent-swatch picker was removed (amber is the default and only accent), and the nav is **optically centered** (a small left nudge balances it against the heavier bordered search box, so it reads centered rather than geometrically dead-centre). All display-only; no backend touched.

reconFTW review + incorporation (2026-07-27)

A reconnaissance engine — reconFTW (six2dez, MIT) — was reviewed end to end for anything worth mining. The short version, and what was folded in:

What it is. An autonomous external-recon / bug-bounty runner: one Bash orchestrator plus eight modules driven by a config file, chaining ~100 tools end to end (optionally across a cloud fleet) and emitting a consolidated report. Its pipeline is OSINT → subdomain enumeration (passive → certificate transparency → ASN/CIDR → permutations → mass-resolve + wildcard-filter → NOERROR → web-metadata scraping → recursive → reverse-IP) → resolution/scope → host/port scan → web probe + screenshots → URL discovery + `gf` triage + JS mining + param discovery + fuzzing → vuln checks → report.

Bottom line: mine the knowledge, never the runner. reconFTW's mechanism — autonomous, fire-and-forget chaining — is the exact opposite of HackPit's one-gated-executor, one-approval-per-command model, so none of the runner, config engine, axiom fleet or monitor/notify machinery is adopted (see Part II's skip list). Its *value* is a battle-tested set of recon tools, proven flag patterns, and a recon ordering — and that lands squarely on HackPit's thinnest area, external attack-surface discovery. So the worthwhile half was folded in as **data and grounding, not a new execution path:**

- **Arsenal (`backend/arsenal/tools.json`)** — **+28 tools, 73 → 101**, in a new `osint` category (6) plus `recon` (10), `web` (10) and `cloud` (2): the URL-triage set, permutation engines, mass-resolution, JS mining, ASN expansion, external OSINT, bucket enum and the injection completers, each with `<target>`-based invocation templates. `executes_nothing` is unchanged, and the arsenal safety suite (schema + inertness + no template hardcodes a host) passes at **101 tools / 257 templates** — a rendered invocation is still a string until a human approves it through the same executor.
- **KB methodology** — **two checklist entries (`recon-methodology-*`)**: the subdomain-enum ordering and the URL→gf-triage→fuzz pipeline, folded in by a new additive ingester (`pipeline/ingest_recon_methodology.py`) that mirrors the corpora discipline — byte-preserving pass-through, its own idempotency marker, re-run byte-identical, and it segregates the corpus block to the tail so `ingest_corpora`'s byte-identity invariant still holds (verified — `test_corpora` green). KB **2,617 → 2,619**.
- **Composer grounding** — an advisory recon-ordering note in the real-target proposer prompt: the sequence as guidance, still one gated command at a time, never a chain.
- **Sandbox image (`docker/Dockerfile.sandbox`)** — the 28 tools installed the way each ships (`go install` for the Go set, venv + PATH wrapper for the Python set, apt for `massdns` / `commix`), with `gf` pattern packs and a static resolver list baked for the no-egress lab. `subwiz` is catalogued but **not baked** — it fetches an ML model at runtime, so it runs only in the egress-enabled sandboxes (engagement / `:kali`), never the airgapped lab; `gotator` / `regulator` cover permutations offline. The ~8–10 GB image was rebuilt and smoke-tested — **all 26 baked tools resolve** in the fresh `hackpit/kali-sandbox:m1` (trufflehog moved to its release-binary installer after a recent release began requiring Go ≥ 1.25; `subwiz` intentionally absent), with the `gf` pattern packs (37) and a 12,956-line resolver list baked in.

The methodology, the arsenal entries, the composer note and the image rebuild are all complete and verified — arsenal suites green at **101 tools / 257 templates**, `test_corpora` byte-identity intact, the full hermetic safety suite green, and the fresh image resolving every baked recon tool.

Persistence / backdoors (TA0003) enrich (2026-07-27)

The post-exploitation **persistence** layer was reviewed the same way — audit first, add only what is genuinely missing — because HackPit already *executes* persistence on scoped targets through the one gated executor (engagement + WinRM + `:kali`) and `msfconsole` is installed. This is an enrich: **knowledge, catalog and describe layers only — no new execution capability, no persistence engine or autorunner.**

What the audit found. The earlier "~60 persistence entries" figure conflated **cloud** persistence with host persistence: of the ~72 persistence-relevant KB entries, **52 are cloud** (IAM / AAD / GCP backdoors) and only 2 were dedicated host-persistence entries. The TA0003 *mechanisms* were present, but only at the command level, scattered through the OSCP/HackTricks corpus (scheduled-task/cron 64 hits, services 59, backdoor accounts 70, web shells 46) — what was missing was the **organised, per-mechanism methodology** tying them together, exactly the reconFTW shape (the raw material existed; the map did not). On the detection side, `attck.py` carried almost none of the persistence technique rows.

- **KB methodology — two persistence checklist entries** (`persistence-methodology-windows`, `persistence-methodology-linux`): a TA0003 mechanism map per OS — Registry Run keys, Startup folder, scheduled tasks, services, WMI subscriptions, accessibility/IFEO, backdoor accounts and web shells on Windows; cron, systemd units/timers, SSH `authorized_keys`, shell-init profiles and backdoor accounts on Linux — each mechanism a single gated command plus the footprint it leaves, cross-referenced to the detection panel. Folded in by a new additive ingester (`pipeline/ingest_persistence_methodology.py`) mirroring the recon-methodology discipline (own idempotency marker, byte-preserving pass-through, corpus block segregated to the tail, re-run byte-identical). KB 2,619 → 2,621; `test_corpora` byte-identity intact. Rootkits and bootkits are named as knowledge only, never tooling.
- **Arsenal** (`backend/arsenal/tools.json`) — +4 tools, 101 → 105, in a new `persistence` category: **SharPersist** (`platform:windows`, add/remove templates, mirroring `rubeus`) plus **Sliver**, **Empire** and **weeveily** as **reference-only** — catalogued but not installed, so reconcile reports them not-present and the planner will not propose them (the same treatment as `subwiz`). Their install and C2 wiring are **deferred to build #4**. `executes_nothing` unchanged; arsenal suites green at **105 tools / 262 templates**.
- **Detection footprint** (`backend/detection/`) — 13 `FootprintSpec`s + 11 ATT&CK rows. `attck.py` gained the persistence technique rows (T1547.001, T1543.003/.002, T1053.003/.006, T1546.003/.004/.008, T1136.001, T1505.003, T1098.004) — additive, transcribed from ATT&CK v19.1 and trimmed to the Windows/Linux/network log-source subset, verified against live upstream. `catalog.py` gained a describe-side footprint per mechanism (Event IDs / Sysmon / Autoruns / auditd, loudness, and a `why_rating` that carries the honesty marker — *quiet = a defender coverage gap, not an operator advantage*). This is the **blue/describe half only**: no OPSEC/evasion notes were added (that channel is build #4), so the `assert_opsec_is_separate` and never-prescribe guards are untouched. Aliases were added for unambiguous binaries only (`schtasks`, `crontab`, `useradd`, `sharpersist`, `weeveily`); multi-purpose `reg` / `sc` / `net` / `systemctl` stay on the `argv` path so a `reg` query is never mislabelled persistence.

All three landed additively and verified — the full hermetic safety suite green (`test_detection`, `test_detection_safety`, `test_arsenal`, `test_arsenal_safety`, `test_corpora`), and `pipeline/detection_sources.py --verify` clean against live ATT&CK v19.1 + SigmaHQ. No image rebuild (OS

built-ins and the installed `msfconsole` cover the mechanisms), and no new execution path — the gated executor already ran these commands, one human approval at a time.

AV/EDR evasion + traffic obfuscation (2026-07-27)

Build #4 closes the C2/evasion channel the persistence enrich deferred. Unlike that enrich, this one **does add execution capability** — a Sliver C2 surface, DNS-tunnel listeners, and a generate-only evasion engine — so the whole design is about making the new surfaces inherit the existing containment rather than sit beside it. It also makes a **deliberate policy change**, recorded as D16 below.

D16 — the OPSEC channel may now prescribe evasion. Until this build, `assert_opsec_is_separate` refused any sensor-blinding phrasing outright: the channel could say "rate-limit and add jitter" but not "patch AMSI". That ban is **lifted entirely and deliberately**. The reason is coherence: this build ships an engine that *emits* AMSI-patch and ETW-blind artifacts, and a tool that produces an artifact it is forbidden to describe is worse than one that describes it honestly. **Two invariants survive and are now the whole contract:** every OPSEC note must carry a substantive `still_recorded` naming what catches the technique anyway, and the blue-view footprint is always produced alongside and is never suppressible. What did **not** change: the blue-side describe-never-prescribe guard (`_evasion_prescription`) is untouched, so the defender's copy is byte-identical to before; and the four execution gates — human approval per command, scope lock, red-confirm, audit — are untouched. This is a **posture shift on a public repo** and is stated plainly rather than buried: HackPit now documents in-process sensor tradecraft, always paired with its detections.

- **Sliver C2 (`backend/cockpit/sliver.py`) — a split-gated surface.** Server lifecycle (start/stop/list) is **human-only** with no red-confirm, mirroring `tunnels.py` : it is operator infrastructure on the operator's own sandbox, so clicking start is the approval. Implant **generation** is a **gated command**, mirroring `session.py` : it builds an `ExecRequest` and runs the real `executor.validate_request` — approval, scope, red-confirm — before anything is produced. It only generates; there is no delivery or execution route, and live beacon catch is deferred. `<listener>` is the operator's callback address and passes through verbatim, never target-substituted. A defect found while building it: `allowlist._FRAMEWORKS` held `"sliver"` , which never matches `sliver-client` , so an implant build would have passed the danger gate with **no red-confirm at all**; the real binary names were added.
- **DNS-tunnel obfuscation (`backend/cockpit/obfuscation.py`) — `dnscat2` / `iodine` listeners**, entirely human-only. The far-side client half is returned as a **string for the operator to carry across by hand**; the module has no delivery primitive and must never gain one. The tunnel's pre-shared key is masked **at source** — `client_command` is built from a masked copy, so the real key never occupies a field that crosses the HTTP boundary. That mattered: the obvious fix (`model_dump(exclude={"secret"})`) still leaks, because the key is embedded verbatim inside the one-liner.
- **Bespoke evasion engine (`backend/evasion/`) — generate-only, with forced honesty.** A top-level package (peer to `detection/` and `state/` , per the cockpit/arsenal decoupling rule). It runs donut/ScareCrow as argv-only `docker exec` into a container resolved from the execution mode — never a request field — and **never runs or deploys what it builds**: the produced artifact is never `argv[0]` , and no deploy primitive exists in the package. **The honest half is computed before anything is built**: if the engine cannot resolve the blue-view footprint for a technique it refuses outright rather than emit a footprint-less artifact. Both the footprint and the `still_recorded` note are required fields on the result model, so no route can return one without the other, and the two PowerShell stubs carry the same honesty header *inside the artifact* so it survives being copied out of HackPit.

- **Detection (backend/detection/)** — 7 new footprint specs and 20 new OPSEC notes. TA0011 gained `c2_dns_tunnel`, `c2_malleable_profile`, `c2_jitter_beacon` and `c2_domain_fronting` (describe-only; domain fronting is documented as largely dead); TA0005/TA0112 gained `evasion_packed_loader`, `evasion_amsi_patch` and `evasion_etw_blind`, which are what the engine maps onto. The 13 persistence specs from build #3 were backfilled with per-mechanism OPSEC notes — 7 new notes alongside the new specs plus those 13 is where the 20 comes from. **SPECS 46 → 53, OPSEC notes 10 → 30** (both re-counted against the live catalog; an earlier draft of this bullet said "4 OPSEC notes", which contradicted its own 10 → 30). Two accuracy corrections were made under review: T1029 had been filed under TA0011 when *Scheduled Transfer* is TA0010 Exfiltration (it was dropped rather than retagged, since an Exfiltration label on a beaconing footprint reads as wrong), and the telemetry strings on five new rows had been written rather than transcribed — replaced with real ATT&CK v19.1 values, `--verify` clean against live upstream. No new ATT&CK rows were needed for the evasion specs: under v19 naming T1562.001 and T1562.006 (Indicator Blocking) *both* revoke into **T1685**, which was already present. An earlier version of this claim paired T1562.006 with T1690 and mapped the ETW footprint onto it — T1690 is the successor of T1562.003 (*Impair Command History Logging*) and covers shell history, not sensors. The Tasks 8–13 review caught it; the spec now cites T1685. `--verify` did **not** catch it and could not have: it checks that a cited id's name, tactic and log sources match upstream, not that the technique is the right one for the activity.
- **Arsenal — 105 → 110 tools**, new `c2` and `evasion` categories; Sliver moved out of `persistence` into `c2`. `<tunnel-zone>` and `<tunnel-net>` are new placeholders rather than reuses of `<domain>`, because the composer's target substitution rewrites any dotted token — spelling them `<domain>` would have pointed an operator's tunnel at the system under test. A latent data bug was found and fixed here: `placeholders` declared `"<payload>"` **twice**, so every `json.load` silently discarded the first description; a duplicate-key check now guards it.
- **Image (docker/Dockerfile.sandbox)** — Sliver, dnscat2, donut and ScareCrow installed; iodine was already present. Three of the plan's assumptions did not survive contact with the base image and were caught by the layer's own smoke tests: `dnscat2` and `ScareCrow` are **not packaged** in kali-rolling, and there is **no Go toolchain** in the image, so release binaries and a from-source dnscat2 build were the only options. Two further faults surfaced only because the smoke test *invokes* each tool rather than running `command -v`: dnscat2's server dies on `require 'ecdsa'` (four gems missing), and `donut-shellcode` is a C extension with no console script, so `python -m donut` cannot work and a real CLI had to be written.

Containment, restated. No orchestrator, agent or loop module can reach any of the three surfaces — asserted by whole-tree source scans, not by convention. Nothing here is autonomous, no gate was weakened, and the only capability the agent gained is none: every new action is human-initiated and audited.

Gate-integrity audit (2026-07-27)

After build #4 shipped, the whole project's **guards** were audited with one question: *would this actually fire?* Not "is a guard present" — the build had already shown that a guard can be present, look correct, pass its tests, and never trigger. **24 guards were probed by constructing a deliberate violation for each. Seven silently failed to fire.** Full detail in `GATE-AUDIT-FINDINGS.md`; the three criticals were:

1. **Eight catalogued tools passed the danger gate with no red-confirm** — demonstrated end to end through `executor.validate_request`. `weevely` (which both generates a webshell *and* drives it), dnscat2's client and server, `iodine` / `iodined`, `commix`, `SSTImap`, `SharPersist` and `Invoke-Obfuscation` all returned zero reasons, as did the argument-driven `sqlmap --os-shell`, `commix --os-shell` and `netexec -x`. **FIXED and pushed (4492fec)**.

2. The WinRM transport classifies only the first token of what is a whole PowerShell script. `executor.py` joins `command+args` and `run_ps()` executes the lot, so `;` and `|` are live separators: `Write-Host go ; Invoke-Mimikatz` is silent, while the same cmdlet as `argv[0]` fires. **FIXED — build #5 (see "Gate integrity — build #5" below)**. This was the most consequential finding of the whole review cycle, because it executes on a real domain-joined host under real credentials.
3. The `:kali` human-only lock never opens 39 of 69 backend modules — its scan globs only `backend/*.py` and `cockpit/*.py`. A planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` passes. The same narrow glob was copied into the tunnels, repeater, terminal and WinRM locks, and the `cockpit→arsenal` lock covers 5 of 22 modules. **FIXED — build #5 (see "Gate integrity — build #5" below)**.

What Critical 1's fix actually changed, because the shape matters more than the list. The `.exe / .py / .ps1` normalisation that the AD sets already did was **collapsed into one shared normaliser used by every set** — that asymmetry was the root cause, not a symptom, and it was why `powershell.exe`, `nc.exe` and even `sliver-client.exe` produced no reason at all on the transport where `.exe` is the ordinary spelling. Two new sets were added rather than padding `_FRAMEWORKS`, so the reason the operator reads is true: `_TUNNEL_TOOLS` (a tunnel is a C2 path and an exfil path, not a payload generator) and `_RCE_TOOLS`. `_FRAMEWORKS` ' `ligolo` entry was corrected to `ligolo-proxy` — the binary the repo actually runs — an entry that could never match, added *after* the `sliver / sliver-client` bug was fixed, and worse than no entry because it reads as coverage.

The finding underneath all three. `test_arsenal_safety` claimed to verify that a catalogued dangerous invocation demands the red-confirm — while testing `python3`, a command that is not in the catalog at all. That single choice is why eight tools shipped ungated. It now pins all 176 catalogued invocation names (every name, alias and template `argv[0]`) into exactly one bucket, so an unclassified tool fails the suite; five planted regressions were run against it and all five fail correctly. Every critical in this audit, and two of the five findings in the Tasks 8–13 review, reduce to the same root cause: **a test that could not fail, or that tested a value the real system never produces**. That is what build #5's third task turns into a written repo convention.

Gate integrity — build #5 (2026-07-27)

The audit's two remaining criticals, and the testing pattern that hid them. Plan: `docs/superpowers/plans/2026-07-27-build5-gate-integrity.md`.

Critical 2 — the WinRM danger gate read the first token of a whole PowerShell script. `cockpit/executor.py` joins `command + args` into one string and the Windows transport hands that string to PowerShell, where `;`, `|` and newlines are live statement separators. The heuristic classified `basename(argv[0])` and scanned only the `args` for markers. So the gate was defeated by moving a cmdlet one token to the right, on the one path that executes against a real domain-joined host under real credentials:

Script	Before
<code>Write-Host go ; Invoke-Mimikatz -DumpCreds</code>	no reason, allowed
<code>Get-DomainUser \ Set-DomainUserPassword</code>	no reason, allowed
<code>IEX (New-Object Net.WebClient).DownloadString(...)</code>	no reason, allowed
<code>Write-Host go + newline + Invoke-Mimikatz</code>	no reason, allowed
<code>powershell -enc <base64></code>	fired by accident — <code>-enc</code> splits into <code>-e</code> , <code>-n</code> , <code>-c</code>

All five became tests first, verbatim, and were confirmed failing before anything was written.

The fix shape, and why it is not a parser. Four options were weighed; the choice was to **scan the whole joined string for every marker, wherever it appears**, and explicitly *not* attempt statement splitting. Correct PowerShell tokenising has to handle quoting, `$ ( )` subexpressions, the `&` call operator, backticks and line continuations — and any parser written here becomes a new bypass surface, which is precisely the bug being fixed: the classifier's model of the input was narrower than the input. Whole-string scanning has no "position" to exploit, and that is the property the first-token design lacked. The cost asymmetry settles it: this gate raises a **red-confirm**, **not a block**, so a false positive costs one click while a false negative runs Mimikatz on a domain controller.

Two things a name-based scan cannot see were closed alongside it. **Download cradles** — `IEX (New-Object Net.WebClient).DownloadString(...)` never spells the payload's name anywhere, so the *cradle* is the marker. **Encoded commands** — `-enc` defeats every text scan by construction, so the payload is decoded and re-scanned, matching every PowerShell prefix of the flag rather than only the spellings a human types; a blob that will not decode is itself reported rather than passed.

The root-cause lock. The bug was never only "the heuristic is too narrow". The string the gate classified and the string the transport executed were **built in two places**, and scanning a different string than the one that runs would have reproduced the bug somewhere new. Both now derive from `executor.join_ps_command()`, and a test asserts on the source — transitively, with a negative control — that they still do. The AD orchestrator's advisory pre-check was moved onto the same union, so the panel can no longer promise a confirm the gate would not require, or stay quiet where it would.

The false-positive cost, measured rather than promised (plan Task 1 Step 6). Run across every AD/Windows template in `tools.json` and every command in the AD graph's edge definitions: **78 of 145 invocations in the full population (53%), and 27 of 43 in the WinRM-reachable subset (62%), now demand a confirm**. The second number is the one that matters — the catalog's Linux impacket invocations go through the docker path, which is untouched. Of those 27, **none is a false positive**: every one is a genuine destructive abuse (Set-DomainUserPassword, Add-DomainObjectAcl, Add-DomainGroupMember, Invoke-Mimikatz, SharPersist, Invoke-Obfuscation, Rubeus ptt, Set-DomainObjectOwner). Read-only enumeration stays silent — Rubeus kerberoast and asreproast, `Get-DomainUser -SPN`, `Find-InterestingDomainAcl`, `Find-LocalAdminAccess`, winPEAS, `Get-ADServiceAccount`, `Get-DomainObject` — and that half is pinned as a control, because a banner on ordinary enumeration is how an operator learns to click through the one that matters. The measurement also found **two real false negatives** the sets had missed: `Invoke-SQLOSCmd` (OS command execution on a SQL host) and `Enter-PSSession / New-PSSession` (a further hop out of an existing session). Both were catalogued; both were silent.

Because whole-string scanning over-flags by design, **every reason names the marker it matched and the offset it matched at** — `powershell script: 'invoke-mimikatz' at offset 14 - dumps/ replicates domain credentials`. A reason the operator can evaluate is a gate; a banner they cannot is decoration, and trading a silent bypass for a decorative one is not a fix.

Critical 3 — eleven source-scan locks, nine of them narrower than their own docstrings. The `:kali` human-only lock is the entire reason the one deliberately-unbounded arbitrary-shell surface is considered safe. Its docstring said *scan the whole (non-venv) source tree*; it globbed `backend/*.py` plus `backend/cockpit/*.py`, which is **30 of 69 modules**, so all of `adgraph/`, `arsenal/`, `codescan/`, `detection/`, `evasion/`, `exploits/` and `state/` were never opened and a planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` — a literal orchestrator module — shipped green. Three distinct defects, now fixed structurally in one shared `backend/test_support/scans.py` rather than by asking each suite to remember:

1. **Narrow file selection** — an `rglob` over the whole backend. Coverage per lock goes 30 to 67 modules; there is no per-suite glob left to get wrong.

2. **Baseline allow-lists** — `{"router.py"}` matched against `f.name` exempted `adgraph/router.py`, `detection/router.py`, `arsenal/router.py` and every other `router.py` in the tree by accident. Allow-lists are now keyed on repo-relative POSIX paths only.
3. **Four literal substrings** — an AST pass alongside them catches what the audit planted and the old predicate missed: aliased imports, imports opened inside a function body, `import_module("cockpit." + "kali")`, `getattr(m, "run_" + "kali")`, and f-strings with constant parts.

Prose is stripped so a module that *documents* a rule does not violate it — this is not hypothetical, documenting the Critical 2 fix tripped the WinRM lock — but ordinary string literals are deliberately **kept**, because a scanner that blanked every string would go blind to `import_module("cockpit.kali")`, which is the indirection the lock exists to catch.

Two more locks were closed with it: the **cockpit→arsenal** lock checked a hardcoded list of five filenames while printing "the cockpit package has zero references to the arsenal" (the package has 22 modules, 17 of which post-date the invariant), and the **evasion agent-path** scan still filtered by *filename*, so `from evasion import engine` in `cockpit/executor.py`, `chat.py` or `adgraph/techniques.py` was never looked at. A module named something nobody predicted is exactly what a name filter cannot cover.

Widening the scans produced two false positives, and both were fixed in the predicate. `cockpit/reconcile.py` "referenced the arsenal" — as a *parameter name*; it takes the loaded catalog as an opaque injected object precisely so it has no import-time dependency. `detection/{catalog,resolver,router}.py` and `report.py` "reached the evasion engine" — they are the *anti*-evasion guard, the code that refuses prescriptive evasion copy. Both locks now assert the claim the invariant actually makes (no import, in any form the AST can see). **Narrowing the file set to silence a false positive is how these guards got broken in the first place**, so that is written into the convention as a rule.

Task 3 — the convention, which is the part that outlasts both fixes. Every critical in this audit, and two of build #4's five review findings, reduce to one root cause: a test that could not fail, or that tested a value the real system never produces. `test_arsenal_safety` claimed to verify that a catalogued dangerous invocation demands the red-confirm while testing `python3`, a command absent from the catalog it was guarding — one choice that hid eight tools. `backend/AGENTS.md` now carries the rule, with the five failures that produced it in a table: **a safety test must iterate real data, assert on what it actually checked, and prove it can fail.** Concretely — draw inputs from the real source of truth so a tool or module added tomorrow is covered by nobody remembering; assert on the *filtered* count, never files opened; and carry a planted-violation positive control in the same test. `test_scans.py` runs **first** in the suite for that reason: ten locks rest on the shared scanner, and it must demonstrate it catches a planted violation before any of them means anything.

Sweeping for the *pattern* rather than the files the plan named found three more, none of which was in the task list. The **live-session stdin lock** — the load-bearing one, since anything able to type into an already-approved running session is executing un-gated commands — said "the whole source tree" and globbed three directories. `test_sliver_safety` and `test_obfuscation_safety`, the two suites the audit called correct, incremented their scanned counter *above* the `test_` skip, so their `>= 40` controls counted files opened rather than files judged. And re-running the audit's own probe list surfaced a real heuristic gap: `sekurlsa::pth ... /run:cmd.exe as argv[0]` fired only because `cmd.exe` happens to sit in its args (change it to `/run:powershell.exe` and it went silent), while `kerberos::list /export` — which writes every ticket in the session to disk — matched nothing at all.

What this build did not close. The tunnel listener start (I2) still bypasses the executor entirely:

`cockpit/tunnels.start_tunnel` runs `docker exec ... ligo10-proxy ...` through `subprocess.Popen` with no `validate_request` call, and `POST /cockpit/tunnels` carries no `approved` or `dangerous_ack` field. The *rewritten*

pivoted command does go through the gated executor, as the module docstring says — but the listener start is the tunnel primitive, and it is ungated. It was outside this plan's scope and is carried forward as an open item. Also unchanged: `iter_run` 's prevalidated path re-checks approval but not danger (nothing is exposed today, because the single `prevalidated=True` caller validates first, but the asymmetry is the kind of thing a future caller trips over), and there is still no route-level authorisation on any of the 113 routes, which remains the known localhost-only posture rather than a new finding.

Verification

- **Hermetic safety suite** (`sh backend/run_safety_tests.sh`) — green after every phase, expanded across all five with `test_phase1_runtime` , `test_state` , `test_scope_hostcheck` , `test_credvault` , `test_corpora` , `test_detection` / `test_detection_safety` (OPSEC channel + blue-view-unchanged), `test_repeater` , `test_tunnels` , `test_report_templates` , persistent-shell containment tests in `test_kali` , Phase 5's `test_terminal` (PTY containment + the sentinel shell provably untouched) and `test_exploits` (version comparison, tiered ranking, executes-nothing), and the Windows backend's `test_winrm` + `test_winrm_safety` (host-locked / no gate bypass / secret never leaks / orchestrator can't auto-run WinRM) with the AD oracle extended to the native Windows variants, plus the post-assessment `test_search_ranking` (substance-gated tier boost + completeness nudge) and `test_codescan_rules` (8-language rule bundle + resolver), plus build #4's six new suites — `test_sliver` / `test_sliver_safety` , `test_obfuscation` / `test_obfuscation_safety` and `test_evasion` / `test_evasion_safety` (no agent path, gated-vs-human-only split preserved, `<listener>` never substituted, the tunnel key never crossing the HTTP boundary, the artifact never executed, and a negative control proving that stripping the OPSEC note makes the engine **refuse** rather than degrade), plus build #5's `test_scans` — the shared source scanner's own regression file, which runs **first** in the suite and plants a violation of each shape (unscanned package, basename-exempted path, aliased import, in-function import, `import_module` string concatenation, f-string) into a temp tree to prove the scanner catches it before ten locks rest on it. **43 test files, 473 checks, 0 failures.**
- **The audit's "probed and holds" list, re-run after build #5** — all 41 items still hold, including the `.exe` spellings from `I1` , the four gate orders, the no-silent-downgrade-to-lab path, and Critical 1's eight catalogued tools. The five demonstrated Critical 2 bypasses are all refused. The planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` now **fails** the scan (verified against a temp mirror; the repo was never modified). Read-only enumeration — `nmap` , `sqlmap --dbs` , `netexec --shares` , `ldapsearch -x` , `hashcat` , `nuclei` — stays clean on both paths.
- **Docker proofs** — `isolation_proof.sh` 4/4 (lab still cannot reach internet or host), `engage_open_proof.sh` 4/4 (engage has full reach).
- **Browser** — every UI surface exercised against a live Ollama backend per the testing rule: the Phase-4 surfaces (payload-set arsenal rows, the OPSEC red-team channel, repeater send/replay/diff, tunnels route preview, exam report templates with the proof table) and the Phase-5 ones — `:terminal` running `top` and `vim` with live resize, `:exploits` resolving `vsftpd 2.3.4` to the backdoor exploit and its CVE, the state panel's per-service jump into it, and `/category/cloud` at 535 entries. **Windows targets** (`/windows`): profile create/list/test/delete and the AD-walk "run on" picker verified against the backend — the connectivity **test** and a live WinRM round-trip are deferred to a real VM (no Windows box exists yet), stated plainly rather than claimed.
- **Frontend** — `tsc` clean, lint at the pre-existing baseline (11 errors + 1 warning, unchanged — verified by stashing the changes and re-running), `next build` exit 0 (routes include `/terminal` , `/exploits` , `/windows`).

Status

Build #4 (AV/EDR evasion + traffic obfuscation) is complete and verified: image `hackpit-kali:build4` builds with all eight new binaries smoke-tested by invocation, the full hermetic suite was green at 42 files / 446 checks at that point, and the frontend builds clean. Tasks 8–13 have now had their independent review (see Part II) — no containment hole, five substantive defects fixed, each with a regression test. **Two caveats are carried forward as open items in Part II rather than papered over**, and build #4 should not be called field-ready until they close: the C2 and tunnel surfaces have never been run against a live beacon, a real delegated DNS zone, or an instrumented Windows host, so their *efficacy* claims are documented rather than demonstrated (the *containment* claims are what the suite locks, and those hold without live infrastructure); and three installs in the image layer are still unpinned, so that layer is not yet byte-reproducible.

A note the review sharpened, because it cuts against the instinct to trust the tooling:

`pipeline/detection_sources.py --verify` reported **0 problems** throughout, including while `evasion_etw_blind` carried a technique id about shell history. It verifies that every id, name, tactic, log source and Sigma reference matches live upstream — it cannot verify that the technique chosen is the right one for the activity being described. A clean `--verify` is a check on transcription, not on judgement.

Build #5 (gate integrity) is complete. All three gate-audit criticals are now closed: Critical 1 in build #4's fix wave (`4492fec`), and Criticals 2 and 3 here. The suite stands at **43 files / 473 checks / 0 failures**, the frontend builds clean, and the audit's full "probed and holds" list was re-run with nothing regressed. What changed is not only two guards but the conditions under which a guard is allowed to be believed: `backend/AGENTS.md` now requires a safety test to iterate real data, assert on what it actually checked, and carry a demonstration that it can fail — and `test_scans.py` runs first in the suite to hold the shared scanner to that standard before ten locks depend on it. **One item from that audit stays open by scope:** `I2`, the ungated tunnel listener start, described at the end of the build #5 section. The remaining work is the deferred list in Part II — chiefly live-fire verification of the C2 and tunnel surfaces, and the three unpinned installs in the image layer.

All five phases are complete, and the Windows execution backend (D9) is now built — the AD attack-path graph executes live over WinRM against an external VMware VM you run. The only remaining work is the deliberately deferred list in Part II — where the largest item, growing tier-1, is writing rather than building — plus the live-box verification of the WinRM driver, which waits on a VM being stood up.